Orbis: Rust vs ASP.NET Core

Optimized EF Core benchmark • results from this test only

Executive summary

Dataset	1,000,003 products • identical SQLite snapshot
Build & protocol	Release build; warmups excluded; persistent HTTP for sequential tests
Concurrency	8 concurrent clients using fresh connections; SHA-256 per response
Repeatability	Two rounds, run in reverse order; figures below average the two round-level metrics
Timing caveat	Client-side response hashing is included in measured request time

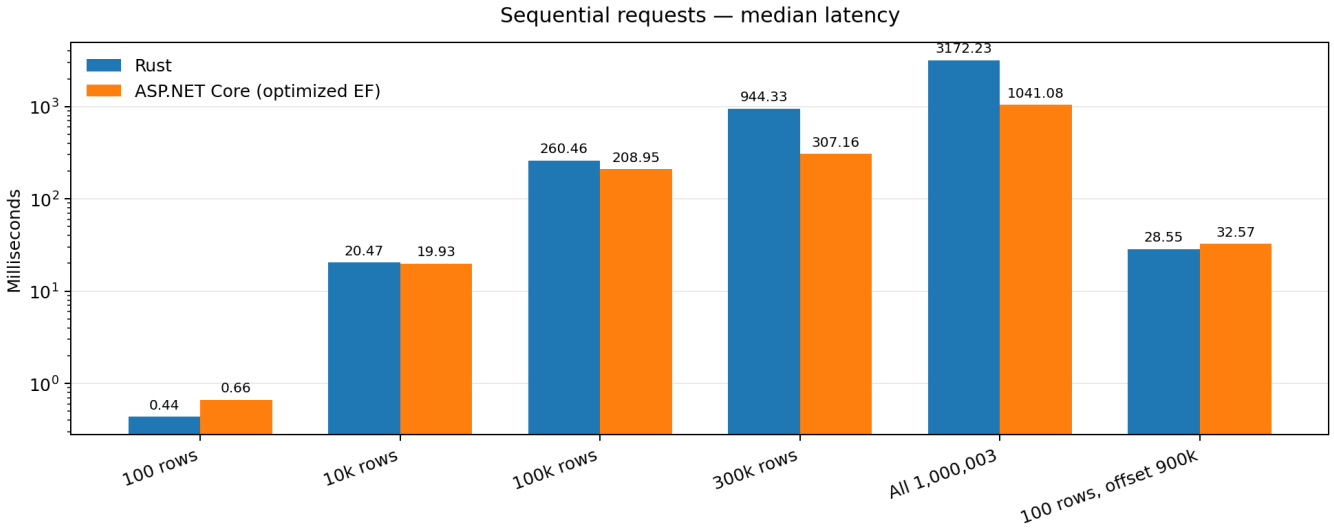
Headline results

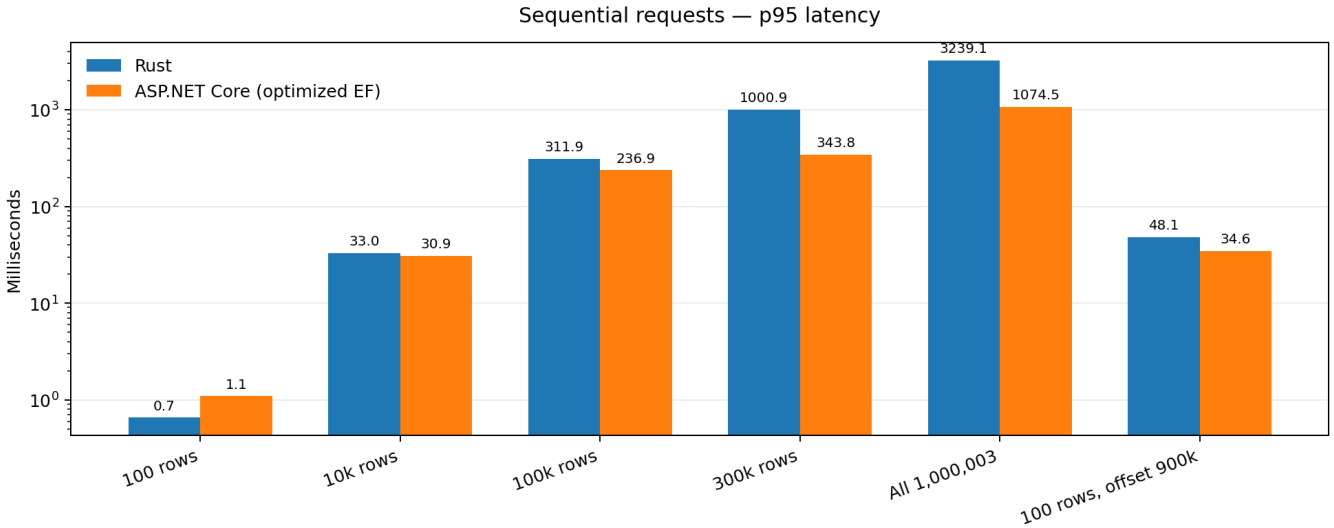
All 1,000,003 rows Rust: 3172 ms ASP.NET Core: 1041 ms	**8 clients • 10k rows** Rust: 13.6 req/s ASP.NET Core: 233.0 req/s	**Warm idle working set** Rust: 11.1 MiB ASP.NET Core: 84.4 MiB

Important interpretation: the EF Core optimization makes this a new result set. The report does not compare it against the previous version, and this benchmark measures a product-list endpoint—not the full ERP workload.

1. Sequential latency

Lower is better. Logarithmic y-axes make small and large payload scenarios readable on the same chart.

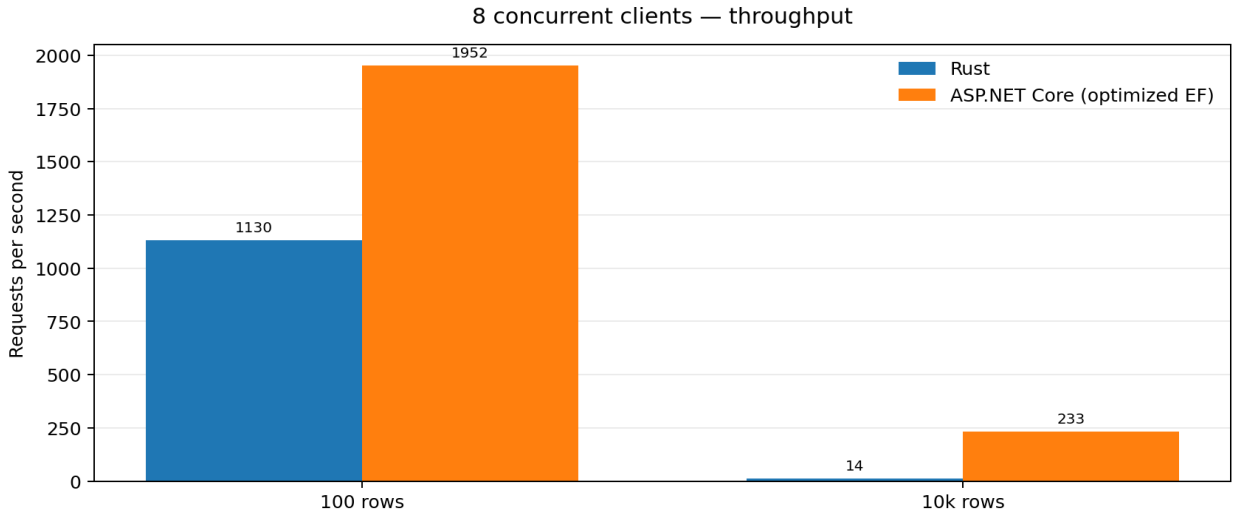


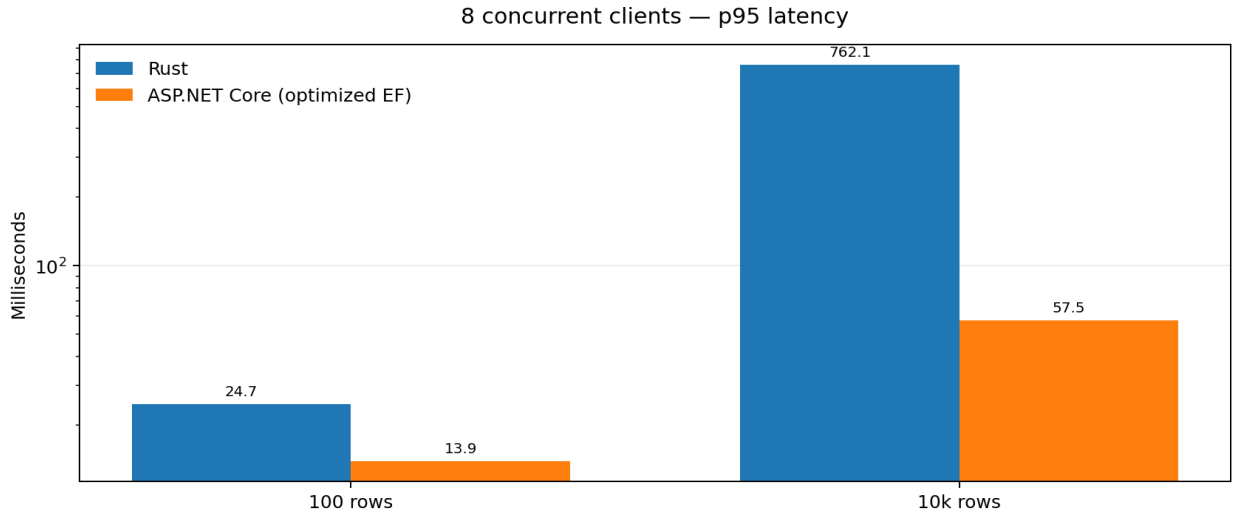


Median latency is nearly tied at 10k rows. ASP.NET Core is faster in the tested 100k, 300k, and all-row responses. Rust is faster for the 100-row request and the high-offset 100-row query by median; its p95 is worse for the high-offset scenario in these averages.

2. Eight concurrent clients

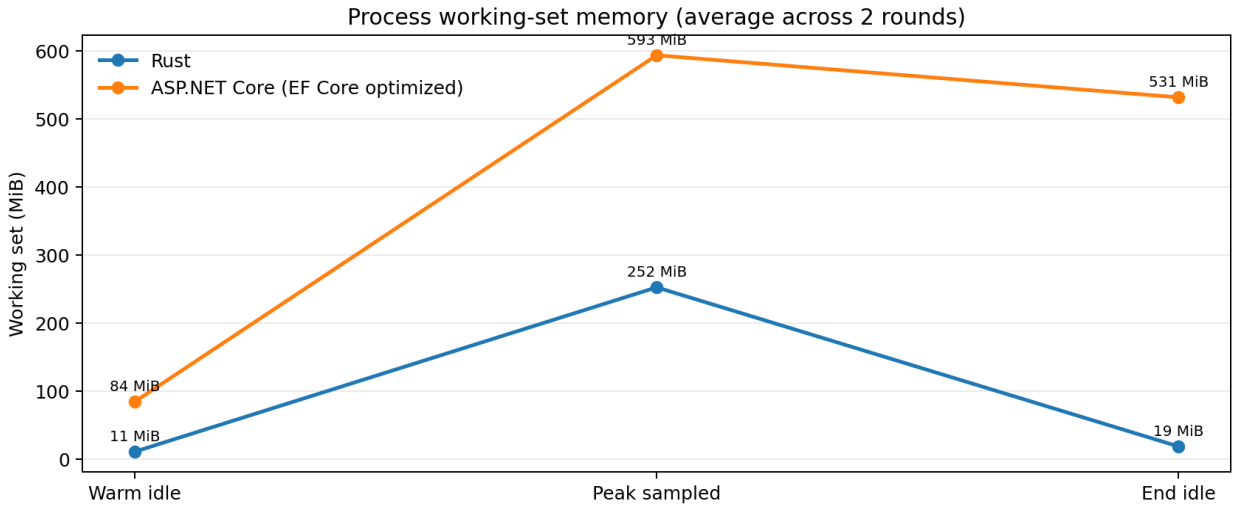
Higher requests/second is better for throughput; lower median/p95 latency is better for responsiveness.





At 100 rows, ASP.NET Core averages about 1951.9 req/s versus Rust 1130.3 req/s. At 10k rows, it averages about 233.0 req/s versus Rust 13.6 req/s (roughly 17.2x the throughput). The 10k result is a major separation and should be reproduced with more rounds and verified against the exact same request counts, payload hashes, and server configuration before treating it as a stable capacity claim.

3. Memory and verdict



Metric (MiB)	Rust	ASP.NET Core
Warm idle working set	11.1	84.4
Peak sampled working set	252.4	593.2
End idle working set	18.7	531.4
Warm idle private memory	5.0	30.0
Peak sampled private memory	332.6	661.9
End idle private memory	13.0	598.4

Verdict for Orbis

Choose ASP.NET Core with the optimized EF Core path as the current default for Orbis. In this test it handles large product-list responses much faster, has higher throughput under 8-client concurrency, and lower p95 latency in the tested concurrent cases. The trade-off is substantially higher process memory—especially retained memory after the test—than Rust.

When Rust makes sense: if very low idle memory and a compact runtime are hard requirements, or for isolated high-throughput services where you can justify the extra development and maintenance cost. It is not the overall performance winner in this optimized EF Core test.

Before making the final architecture decision: repeat the concurrency test several times; measure CPU, allocations/GC, and DB wait time; test writes and realistic ERP queries (joins, filtering, sorting, transactions, reports); and benchmark paginated endpoints. Avoid serving a million-row JSON array to a browser as a normal UI flow. This benchmark alone cannot prove which language is best for the whole ERP.

Memory values are averages of two sampled process-level runs. Peak sampled memory is not per-request allocation, and high end-idle memory can reflect retained heap/cache behavior; profile before concluding it is a leak.